Agentic Multimodal RAG with Grounded Explanations for Complex Scientific Reasoning

October 22, 2026

Abstract

Researchers increasingly rely on AI systems to answer questions about scientific papers, whose key results sit in tables and figures. Single-pass multimodal retrieval-augmented generation (RAG) answers every question, including those its evidence does not support, and shows no proof a reader could check. This thesis builds and evaluates an agentic multimodal RAG system that fetches text, tables and figures over several rounds, checks before writing whether its evidence suffices, verifies every claim against the evidence it cites, and marks each claim's proof with a box on the page. The system runs on consumer hardware with an open 4-bit 7B vision-language model. On 150 questions from the SPIQA benchmark, judged against gold answers by a model judge that agreed with a human grader on 22/25 answers (kappa 0.72), the agent was wrong on 43% of the questions against 66% for a single-pass baseline, but also correct less often (21% against 34%), because its abstentions did not single out the hard questions. Behind the identical graph, a stronger model made abstention selective. When a claim cited the right table or figure, its proof box pointed at the hand-annotated answer 71% of the time, and every checked table-cell proof did. The results show that the gates reduce confident errors and that their benefit is limited by the capacity of the model that makes their judgements.

Contents

1 Introduction **4**

1.1 Problem Statement 4
1.2 Research Questions 5
1.3 Research Objectives 5
1.4 Contributions 5

2 Related Work **7**

2.1 Multimodal Document Question Answering 7
2.2 Visual Document Retrieval 7
2.3 Agentic and Corrective Retrieval-Augmented Generation 8
2.4 Attribution and Evaluation of Generated Answers 8

3 Methodology **9**

3.1 The Central Idea 9
3.2 Agentic System Architecture 10
3.2.1 The System as a State Machine 10
3.2.2 Query Decomposition 10
3.2.3 The Reasoning Loop 11
3.2.4 The Evidence Sufficiency Gate 11
3.2.5 The Faithfulness Gate 12
3.3 Modality-Specific Retrieval Tools 12
3.4 Cross-Modal Evidence Fusion 13
3.5 Model Choice 14
3.6 Grounded Multimodal Explainability 15
3.7 Experimental Setup 15
3.7.1 Dataset 15
3.7.2 Systems Compared 16
3.7.3 Outcomes and the Accuracy Judge 17

3.7.4 Metrics . . . 17
3.7.5 Hand Annotation . . . 18

4 System Architecture **19**
4.1 Overview . . . 19
4.2 Ingestion and Indexing Layer . . . 21
4.3 Agentic Planner . . . 21
4.4 Evidence Sufficiency Gate . . . 22
4.5 Grounded Generator and Faithfulness Gate . . . 22
4.6 Attribution Layer . . . 22
4.7 Mapping to Research Objectives . . . 23

5 Graph Architecture and Node Specification **24**
5.1 Design Rationale . . . 24
5.2 Node-by-Node Specification . . . 26
5.2.1 Ingestion Path (N3–N4) . . . 26
5.2.2 Query Encoding Path (N1–N2) . . . 26
5.2.3 Planning and Retrieval (N5–N7) . . . 27
5.2.4 Evidence Buffer and Sufficiency Gate (N8, G1) . . . 27
5.2.5 Generation and Faithfulness Gate (N9, G2) . . . 28
5.2.6 Attribution and Output (N10–N12) . . . 28
5.3 Control Loops . . . 29
5.4 Mapping to Research Objectives . . . 29

6 Implementation **31**
6.1 Software and Hardware . . . 31
6.2 Ingestion and Indexing (N3, N4) . . . 32
6.3 The Language Model in the Loop . . . 32
6.4 Generation and Verification (N9, G2) . . . 33
6.5 Exact Proof Boxes (N10b) . . . 33
6.6 Evaluation Runner and Power-Cut Safety . . . 33
6.7 Design Validation on a Synthetic Paper . . . 34

7 Results **36**
7.1 Overview . . . 36
7.2 RQ1: Does the Agentic Loop Improve Accuracy and Reduce Hallucination? . . . 37
7.3 A Change Made During the Evaluation . . . 39
7.4 RQ2: Can Visual Retrieval Fetch the Right Tables and Figures? . . . 39
7.5 RQ3: Can the Agent Produce Precise Boxes? . . . 40

7.6 A Stronger Model Behind the Same Graph 41

8 Discussion **42**
8.1 The Model, Not the Architecture, Is the Bottleneck 42
8.2 What the Development Error Analysis Showed 43
8.3 Measuring Grounding 43
8.4 Practical Constraints 44
8.5 Threats to Validity 44

9 Conclusion **45**
9.1 Contributions 46
9.2 Future Work 46

Bibliography **47**

Chapter 1

Introduction

Researchers increasingly rely on AI-assisted literature review systems powered by Large Vision-Language Models (LVLMs) to synthesize information. However, scientific papers are highly multimodal. Vital findings are distributed across interleaved narrative text, dense statistical tables, and experimental plots. Comprehending these documents requires multi-hop reasoning across these distinct modalities. The motivation of this research stems from the urgent need to bridge the gap between text-only language models and the deeply multimodal reality of scientific research.

1.1 Problem Statement

Current multimodal Question Answering (QA) and Retrieval-Augmented Generation (RAG) [17] systems fail on complex scientific literature due to two main reasons:

1. **Passive Retrieval:** Existing pipelines retrieve context in a single pass. They lack the iterative agency to perform multi-step cross-modal navigation (e.g., jumping from a text claim to a supporting table), leading to severe factual hallucinations.

2. **The Explainability Gap:** Current systems output answers with no verifiable visual grounding. They cannot point to the specific table cell or chart region used as evidence, making their generated reviews untrustworthy for academic use.

1.2 Research Questions

To address these limitations, this research is guided by the following questions:

- **RQ 1:** How does an iterative, agentic reasoning loop improve factual accuracy and reduce hallucination on the SPIQA dataset [20] compared to single-pass retrieval?
- **RQ 2:** How effectively can visual document embeddings (e.g., ColPali [10]) be utilized by a multimodal agent to fetch necessary tables and figures?
- **RQ 3:** Can the agent generate precise visual attribution (bounding boxes) for its answers, and how does this affect human verifiability?

1.3 Research Objectives

Mapping to the research questions, this work aims to achieve the following:

- **Objective 1:** Develop a Multimodal Agentic Planner that iteratively fetches text, tables, and figures.
- **Objective 2:** Integrate a vision-document retriever to dynamically fetch ground-truth visual data from PDFs.
- **Objective 3:** Implement an Evidence Sufficiency Gate to verify proof before answering, and a Faithfulness Gate to check the drafted answer against that proof, mitigating hallucinations.
- **Objective 4:** Engineer the output layer to generate spatial bounding-box coordinates on charts and tables to ensure verifiability.

1.4 Contributions

By bridging visual document retrieval with iterative agentic reasoning, this thesis:

- **Builds the system end to end** on consumer hardware with an open model: a planner over text, table and figure tools, a Sufficiency Gate, a Faithfulness Gate that verifies every claim's citation, and exact proof boxes with an execution trace.

- **Reduces confident errors:** on 150 SPIQA questions the agent was wrong far less often than a single-pass baseline (43% against 66%), at the cost of abstaining on questions the baseline answered correctly.
- **Shows where the benefit is limited:** holding the architecture fixed and changing only the model, abstention became selective with a stronger model, so the gates' benefit depends on the model that makes their judgements.
- **Mitigates the trust gap:** when a claim cites the right table or figure, its proof box points at the hand-annotated answer 71% of the time, and every checked table-cell proof does.

Chapter 2

Related Work

This work draws on four lines of research: question answering over documents with tables and figures, visual document retrieval, agentic and self-correcting retrieval-augmented generation, and the attribution and evaluation of generated answers.

2.1 Multimodal Document Question Answering

Document question answering moved from text to the visual content of documents with DocVQA [19], which asks questions about images of document pages, and ChartQA [18], which requires reading values and trends from charts, often with arithmetic. SPIQA [20] brings this to scientific papers: its questions are written so that the answer depends on a figure or table of the paper, each annotated with its reference. These benchmarks evaluate whether an answer is correct; none asks a system to show where on the page its answer comes from, which is the gap this thesis addresses with proof boxes.

2.2 Visual Document Retrieval

Late-interaction retrieval, introduced for text by ColBERT [15], scores a query against a document by matching each query token to its most similar document token. ColPali [10] applies it to page images: a vision-language model embeds each page as a grid of patch vectors, so pages can be retrieved without OCR, and its ColQwen2 variant is used here. VisRAG [26] and M3DocRAG [5] build

retrieval-augmented generation on such page retrievers, passing retrieved page images to a vision-language generator. These systems retrieve once and answer; the present work retrieves pages and regions repeatedly under a planner, and reuses the retriever's patch similarities only as a last resort for locating a proof.

2.3 Agentic and Corrective Retrieval-Augmented Generation

Retrieval-augmented generation [17] conditions a generator on retrieved passages. ReAct [25] interleaves reasoning with tool calls, so that each observation informs the next action, which is the pattern of the planner here. Self-RAG [2] trains a model to decide when to retrieve and to critique its own output with reflection tokens, and Corrective RAG [24] adds an evaluator that grades retrieved documents and triggers further retrieval when they are poor. The two gates of this thesis play similar roles — one judges the evidence before writing, one judges the claims after — but are realised by prompting an off-the-shelf model, with no training, and they operate across modalities. Selective question answering [14] studies when a model should abstain; the selectivity analysis of Chapter 7 applies its central question to the agent's abstentions.

2.4 Attribution and Evaluation of Generated Answers

ALCE [11] asks language models to cite the passages that support each statement and measures citation quality; the claim links of this thesis follow the same idea but verify every citation before publishing it. RAGAS [9] evaluates retrieval-augmented answers with a language model as the judge, and the reliability of such judges has been studied directly [28]; this thesis uses a model judge and checks it against a human grader. For visual attribution, saliency methods such as integrated gradients [23] explain a model's input attention, and the pointing game [27] evaluates whether a saliency map's peak falls on the annotated object; here the same test is applied to proof boxes.

Chapter 3

Methodology

This chapter describes how the proposed system works and how it was evaluated. The presentation is deliberately ordered from intuition to formalism: each mechanism is first described in plain terms, then stated precisely. Every equation is followed by a reading of what it says in words, so that the formal notation documents the design rather than substituting for an explanation of it.

3.1 The Central Idea

A conventional Retrieval-Augmented Generation system behaves like a student who visits the library exactly once. Whatever material is gathered in that single trip is all the material available when the essay is written. If a claim turns out to need a supporting table that was not collected, the student cannot go back — and the most likely outcome is that the missing number is invented rather than the gap admitted.

The proposed system behaves instead like a careful researcher. It gathers some evidence, stops to ask whether that evidence is actually enough, returns for more when it is not, and refuses to write until the question can be answered from what it has in hand. Three mechanisms make this behaviour concrete:

1. An **agent** that decides, at each step, which kind of evidence to fetch next — and may fetch repeatedly rather than once.

2. An **Evidence Sufficiency Gate** that stands between gathering and writing, and only opens when the gathered evidence can support an answer.

3. A **Faithfulness Gate** that checks the written answer against the gathered evidence, and sends it back for rewriting if any claim is unsupported.

The remainder of this chapter formalises these mechanisms, describes the retrieval tools available to the agent, explains how evidence from different modalities is combined, and defines the evaluation plan.

3.2 Agentic System Architecture

3.2.1 The System as a State Machine

The agent is built on the ReAct paradigm [25], in which a model alternates between reasoning about what it needs and acting to obtain it. Formally, the system is a state machine:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{O}, \mathcal{E} \rangle \tag{3.1}$$

In words, the system is fully described by five things: what it currently knows, what it is able to do, how doing something changes what it knows, what it gets back when it acts, and how it decides when to stop. Each component is defined below.

$\mathcal{S}$ — **the state.** Everything the system currently knows. This comprises the interpreted intent of the query, the record of which retrievals have already been performed, and the evidence buffer $\mathcal{B}_t$ holding every piece of evidence gathered so far.

$\mathcal{A}$ — **the action space.** The set of tools the agent may invoke. Each tool retrieves a different modality.

$\mathcal{T}$ — **the transition function.** How the state changes once a tool has been used: new evidence is added to the buffer and the retrieval history is extended.

$\mathcal{O}$ — **the observation space.** What comes back from a tool — a text passage, a parsed table, or a page region.

$\mathcal{E}$ — **the Evidence Sufficiency Gate.** The decision procedure that determines whether gathering should continue or generation may begin.

3.2.2 Query Decomposition

Given a complex scientific query Q, the agent does not attempt to satisfy it in one retrieval. It first breaks the query into a sequence of smaller, individually

answerable sub-goals:

$$Q \longrightarrow \{q_1, q_2, \ldots, q_k\} \tag{3.2}$$

For example, the query *"Does the accuracy gain reported in Section 4 hold for the low-resource subset?"* decomposes into two sub-goals: first locate the claim about the accuracy gain in the narrative text, then locate the table that reports results broken down by subset. Decomposition matters because the second sub-goal cannot even be formulated until the first has been answered — it is the retrieved claim that tells the agent which table to look for. This is what is meant by *multi-hop* reasoning, and it is the reason a single retrieval pass is structurally incapable of answering such questions.

3.2.3 The Reasoning Loop

At each time step t the agent selects one action a_t from the action space:

$$\mathcal{A} = \{\texttt{TextRetriever}, \texttt{TableRetriever}, \texttt{FigureRetriever}\} \tag{3.3}$$

A fourth tool, `RegionZoom`, is not chosen by the agent: it runs automatically after the `FigureRetriever` whenever the retrieved figure covers less than 30% of its page, because a small figure cannot be read at page resolution. Executing a_t returns an observation o_t, which is appended to the evidence buffer. After t steps the buffer therefore contains the accumulated result of every retrieval performed so far:

$$\mathcal{B}_t = \bigcup_{i=1}^{t} o_i \tag{3.4}$$

The union notation simply says that nothing is discarded: evidence retrieved at step 1 is still available at step 5. The agent then consults the Sufficiency Gate, and either loops back to select another tool or proceeds to generation.

3.2.4 The Evidence Sufficiency Gate

The Sufficiency Gate is the principal novelty of the architecture, and its purpose is best stated as a question the system asks itself before writing anything: *"Can I answer this from what I have actually retrieved?"*

Concretely, a meta-evaluator inspects the query Q alongside the current buffer $\mathcal{B}_t$ and returns a judgement of sufficiency. Consider the running example. After the first retrieval the buffer contains the narrative sentence asserting an accuracy gain, but nothing about the low-resource subset. The claim is present; the evidence qualifying it is not. The gate therefore fails, the agent reformulates and

dispatches the `TableRetriever`, and the gate is re-evaluated against the enlarged buffer.

The significance of this design is that it is *preventative*. A conventional pipeline would forward the incomplete buffer to the generator and depend on the generator to notice what was missing — which is precisely the situation that produces fluent, confident, unsupported answers. The gate removes the opportunity to make that error rather than attempting to detect it afterwards.

The loop is bounded by a maximum of three retrieval rounds. If sufficiency is not reached before the bound is exhausted, the system distinguishes two cases. If the gate found no fact that answers the question, the system reports that the evidence is inadequate rather than answering anyway: a researcher can act on a stated gap, but cannot detect a fabrication. If the gate did find answering facts but was not fully convinced, the system answers from those facts as a *best effort*: every claim still passes the Faithfulness Gate, and the answer carries the visible flag "the evidence found may be incomplete". Section 7.3 explains why this second case was added.

3.2.5 The Faithfulness Gate

The Sufficiency Gate guarantees that the evidence needed for an answer is present; it does not guarantee that the generator uses only that evidence. A second gate therefore runs after generation. The draft answer is split into individual factual claims, and each claim is checked against the evidence buffer $\mathcal{B}_t$. If any claim is not supported by an item in the buffer, the gate fails and the generator is asked to rewrite the unsupported part using only buffered evidence.

Because the generator has no access to the corpus other than through $\mathcal{B}_t$, this check is bounded: each claim is compared against a fixed, finite set of retrieved items rather than against the world at large. Like the retrieval loop, the rewrite loop has a maximum iteration count; if it is exhausted, unsupported claims are removed and reported as unconfirmed rather than kept.

3.3 Modality-Specific Retrieval Tools

Scientific PDFs are not uniform documents, and a single retrieval mechanism handles them poorly. The action space therefore provides four tools, three matched to a different kind of content and one for inspecting a page in finer detail. Each is described below together with the reason it exists as a separate tool.

`TextRetriever`. Retrieves paragraphs of the paper's narrative text by combin-

ing two rankings with reciprocal rank fusion [7]: dense semantic retrieval with SPECTER2 [21], a model built on SciBERT [4] and trained for scientific document retrieval, and exact keyword overlap weighted by inverse document frequency. The keyword ranking was added because, within a single paper, every paragraph shares the topic that SPECTER2 encodes, while SPIQA questions reuse the paper's own terms; on the development questions keyword search alone found a relevant paragraph more often than SPECTER2 alone, and the fusion put a correct paragraph first most often.

`TableRetriever`**.** Works in two steps. First, it ranks the paper's tables by the words of their captions and by the visual (ColQwen2) score of the page they sit on. Second, it runs a table-structure recogniser, such as the Table Transformer [22], on that region; this recovers rows, columns and a bounding box for every cell, and each cell's text is read from the PDF's text layer at that position. The result is a normalised table *that preserves the two-dimensional coordinates of every cell*. Coordinate preservation is not incidental detail. The bounding boxes required by Objective 4 can only be produced if the spatial layout of the table survives parsing; a parser that returned a table as flat text would satisfy the reasoning requirement and silently destroy the attribution requirement.

`FigureRetriever` **(ColQwen2).** Embeds entire PDF pages as grids of visual patches using ColQwen2, a member of the ColPali family [10] of vision-language retrievers that scores pages by late interaction between query tokens and page patches [15], locating charts, plots and diagrams without depending on OCR. This matters because OCR fails most severely exactly where scientific documents are most information-dense — multi-column headers, merged cells, and axis labels. Treating the page as an image sidesteps that failure mode entirely.

`RegionZoom`**.** Re-renders a figure already returned by the `FigureRetriever` at 300 dpi (at most 1024 pixels on its long side). A page-level match identifies which page holds a chart, but not which data point answers the question; zooming supplies that finer level of detail. It runs automatically for figures that cover less than 30% of their page.

3.4 Cross-Modal Evidence Fusion

Once evidence has been retrieved from different modalities, the system must relate the pieces to one another. A symbol appearing in a sentence and the same

symbol appearing in a chart legend must be recognised as referring to the same quantity; otherwise the system holds two disconnected facts rather than one linked piece of evidence.

This linkage is performed in two places, neither of which requires training a new model component.

1. **Explicit linking in the evidence buffer.** Every item added to $\mathcal{B}_t$ carries its provenance: source paper, page, retrieving tool, and step. Items are additionally linked when they share a page or when one explicitly references the other — for example, a sentence containing "Table 3" is linked to the table whose caption begins "Table 3". The buffer is therefore a small graph of connected evidence rather than an unordered list.

2. **Joint reasoning in the generator.** The linked items are passed to the vision-language generator as a single interleaved context, with each text passage placed next to the table or figure crops it is linked to. The generator's own attention layers then relate text tokens to image regions — this is the joint text–image reasoning such models are pre-trained to perform, and no additional fusion layer is added on top of it.

The practical consequence is that downstream generation is conditioned on linked multimodal evidence rather than on two independent evidence streams that happen to have been retrieved together.

3.5 Model Choice

The language model is Qwen2.5-VL-7B-Instruct [3], an open-weight vision-language model, quantised to 4 bits and served locally with llama.cpp [12]. Three reasons decided this choice. It runs entirely on the available hardware (one consumer GPU with 12 GB of memory), so no evaluation depends on a paid service. Its weights are fixed, so every reported number can be reproduced. And it reads images, which the figure and table tools require. The same model serves as the question analyser, the planner, the writer, and the judge behind both gates. A stronger hosted model (Gemini 3.5 Flash-Lite, free tier) was run behind the identical graph as a comparison, to separate the contribution of the architecture from that of the model.

3.6 Grounded Multimodal Explainability

A system that produces a correct answer without showing where it came from remains unusable for academic work, because a reader has no way to check it. The generation phase therefore emits three audit artefacts alongside the natural-language response. Each answers a different verification question.

1. **Claim Links** — *"Which evidence did this come from?"* Every claim the writer produces must cite the evidence it rests on: a verbatim quote from a paragraph, a cell of a parsed table (row, column and value), or an observation about a figure. The Faithfulness Gate verifies each citation, so a published claim is linked only to evidence that was checked to contain it. This replaces the token attribution by integrated gradients [23] first planned: gradients cannot be computed through a quantised model served by llama.cpp, and a verified citation states the source more directly than a saliency score.

2. **Visual Grounding (Bounding Boxes)** — *"Where on the page is the proof?"* Explicit coordinates $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$ mark the exact spot a claim rests on. Exact boxes are preferred whenever the page offers them: the cell coordinates preserved by the `TableRetriever`, a bar of a vector chart read from the PDF, the quoted sentence found on the page, the caption, or the diagram labels an observation names. Only for photographs, which have no printed words, is the box obtained from ColQwen2's patch-similarity map, computed for the claim's own text, by taking the enclosing box of the highest-scoring patches. This is the artefact that discharges Objective 4.

3. **Retrieval DAG Trace** — *"How did the system get here?"* A logged execution graph recording which tools were called, in what order, and how many times each gate rejected the buffer or the draft before opening. This makes the reasoning path auditable rather than merely asserted.

Together these mean a reader need not trust the system's answer. They can check it against the highlighted sentence, look at the boxed table cell, and inspect the path by which both were found.

3.7 Experimental Setup

3.7.1 Dataset

The framework was evaluated on **SPIQA** [20], a dataset of questions over computer-science papers whose answers depend on the papers' figures and tables. Each

question is annotated with its reference figure or table, which provides figure-level ground truth for retrieval. The questions were drawn from SPIQA test-A (118 papers, 666 questions) with a fixed random seed (42). Papers, not questions, were assigned to splits, so that no paper contributes questions to both:

- a **development split** of 20 questions from 5 papers (9 table, 11 figure), used for every design decision, prompt and threshold;
- a **test split** of 150 questions from 31 papers (71 table, 79 figure), used once for the final evaluation, with at most 8 questions per paper so that a few question-heavy papers cannot dominate.

Before the test split was run, every setting of the system was written to a frozen configuration file, and the evaluation runner refuses to start on the test split unless the live settings equal it. One exception is documented in Section 7.3.

SPIQA does not annotate *regions* within a figure or table. Evaluating box precision therefore required a manually annotated subset: 50 test questions (25 with a table and 25 with a figure as reference), drawn at random before any system output was inspected, for which the part of the table or figure that holds the answer was boxed by hand (Section 3.7.5). Since not every SPIQA question is multi-hop, results are also reported on a **multi-hop subset**: the test questions that ask for a reason or an implication of a result ("why", "suggest(s)", "possible explanation"), whose result is in a table or figure and whose reason is in the text. The subset was chosen by this fixed rule over the question text alone and contains 10 questions.

3.7.2 Systems Compared

All systems use the same language model, the same retrieval tools, the same prompts and the same output layer; each removes one part of the design.

- **Full**: the complete agent — question analysis, the planner choosing one tool per round (at most three rounds), both gates, the rewrite loop (at most two rewrites), and best effort after the retrieval budget.
- **Single-pass**: no planner, no loops and no gates. Every tool runs once with the question, and the writer answers from what came back. Comparing Full with Single-pass isolates the contribution of *agency*.
- **Text-only agentic**: the full graph with the text tool only; tables and figures are available only as the plain text extracted from them. Comparing Full with Text-only isolates the contribution of *seeing* tables and figures.

- **Full, strict G1** (ablation): the full system as first frozen, which abstains whenever the Sufficiency Gate is not convinced after three rounds (Section 7.3).
- **Full, Gemini**: the full system with Gemini 3.5 Flash-Lite behind every model call instead of the local 7B model, on the free tier. It separates what the architecture contributes from what the model contributes.

3.7.3 Outcomes and the Accuracy Judge

Accuracy alone hides the behaviour this work is about, so every question has one of three outcomes. A question is **answered** when at least one claim reaches the reader; it is then **correct** or **wrong**. It is **abstained** when the system reports that it could not confirm an answer, or when the Faithfulness Gate withheld every claim. A wrong answer is the hallucination this work tries to prevent: a system that turns wrong answers into abstentions is preferable for a researcher, who can act on a stated gap but cannot detect a confident error.

Answers were judged against SPIQA's gold answer by the local model, prompted to call an answer correct when it gives the gold answer's main point (the value, name, trend or comparison asked for) and states nothing that contradicts it. The judge writes its reason before its verdict. To check the judge, 30 answers (6 from each system, shuffled) were graded by a human annotator who saw neither the system nor the judge's verdict, under the same rule.

3.7.4 Metrics

Outcome rates. The shares of correct, wrong and abstained answers, and the precision when answering (correct / answered).

Retrieval Faithfulness (RF). The share of the distinct tables and figures in the evidence buffer that are the question's reference. It is undefined for the text-only system, which retrieves no region. The **hit rate** reports whether the reference was in the buffer at all. *RQ1, RQ2.*

Modality Attribution Precision (MAP). Planned as a hand check of whether each published claim cites the right source. For lack of time it was replaced by an automatic stand-in: the share of published claims that cite the reference table or figure. The stand-in undercounts claims that rightly cite supporting text. *RQ2.*

Agent Efficiency Score (AES). Hops needed (1, or 2 on the multi-hop subset) divided by search-tool calls, capped at 1. The automatic zoom is not a choice and is not counted. *RQ1.*

Cross-modal Grounding Score (CGS). Per annotated question, the best intersection over union (IoU) between any proof box and any hand-drawn answer box on the same page; a question without a box scores 0. Reported over all annotated questions, over answered ones, and as the share with IoU ≥ 0.5. Because IoU proved to penalise correct proofs drawn at a different size (Section 7.5), the **pointing game** [27] is also reported: a proof hits when the centre of its box lies inside a hand-drawn box. This second measure was added after the IoU results were seen. *RQ3.*

Every rate is reported with a 95% bootstrap interval over questions [8] (10,000 resamples). Differences between two systems are computed on the same questions, in the same order, and called beyond luck when their interval excludes zero.

3.7.5 Hand Annotation

The boxes and the 30 judge-check grades were produced by Saad Al Abeed, a co-author (Computer Science and Engineering, Islamic University of Technology), with an offline annotation page that showed each question, its gold answer, and the page with the reference table or figure outlined. Written guidelines asked for a box around only the part that holds the answer (the cell, the bar or line segment, the diagram block). Two rounds of quality control narrowed 16 boxes that covered the whole table or figure although the question named specific values; these were selected by a fixed rule and without looking at any system's boxes. In the first round (9 boxes) the boxes were redrawn by the first author on the values that the gold answer names; in the second (7 boxes) by the annotator. Five whole-figure boxes remain, for questions about an entire figure (the steps of a pipeline, a trend across a chart).

Chapter 4

System Architecture

The proposed system is organized as a five-stage pipeline: an ingestion and indexing layer, an agentic planner, an evidence sufficiency gate, a grounded generator with a faithfulness gate, and an attribution layer. Unlike passive single-pass retrieval pipelines, the planner and sufficiency gate form a closed feedback loop, allowing the system to iteratively gather additional evidence before committing to an answer; the generator and faithfulness gate form a second loop that removes unsupported claims from the draft answer. Chapter 5 refines each stage into individual graph nodes.

4.1 Overview

Figure 4.1 illustrates the overall data flow. A user query enters the Agentic Planner, which issues retrieval actions against a dual index built during ingestion. After each retrieval round, the Evidence Sufficiency Gate evaluates whether the collected evidence is adequate to answer the query. If not, control returns to the planner for another retrieval iteration; if so, the system generates an answer from the evidence alone. The Faithfulness Gate then checks every claim in that answer against the evidence, returning it for rewriting if needed, before attribution is produced.

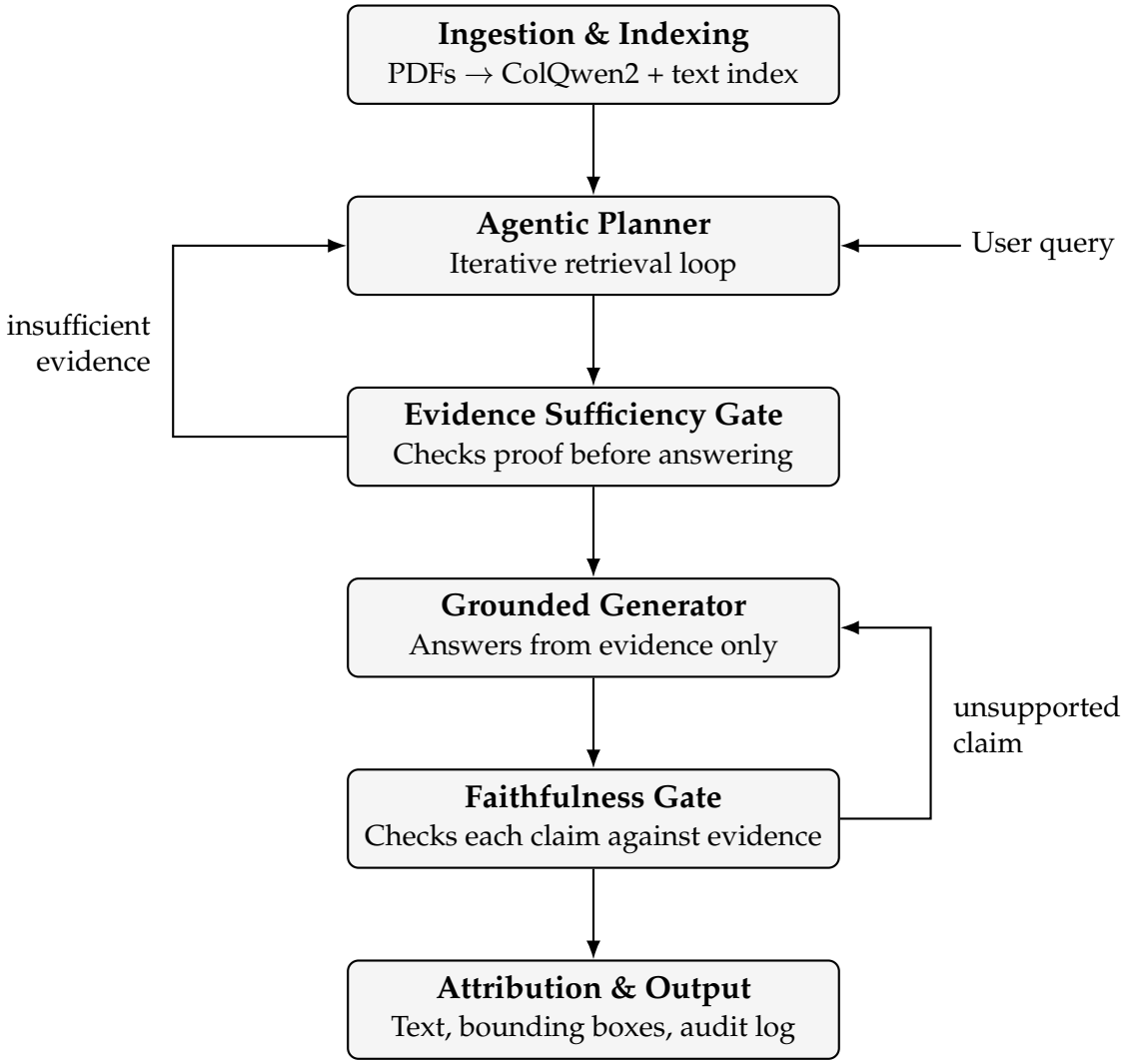


Figure 4.1: High-level architecture of the multimodal agentic literature review system.

4.2 Ingestion and Indexing Layer

Each PDF page in the corpus is rasterized into a page image, which is embedded using a visual document retriever (ColQwen2, of the ColPali family [10]) to produce patch-level embeddings without requiring OCR or layout parsing. In parallel, narrative text is extracted and segmented into chunks, which are embedded using a retrieval-trained scientific text encoder (SPECTER2 [21]) and stored in a FAISS [13] index. This produces two complementary indexes:

- **Visual index:** Page-level patch embeddings for retrieving tables, figures, and dense visual content.
- **Textual index:** Chunk-level embeddings for retrieving narrative claims and discussion.

4.3 Agentic Planner

The Agentic Planner is an LVLM-driven controller that operates an iterative reasoning loop (Thought $\rightarrow$ Action $\rightarrow$ Observation), inspired by the ReAct paradigm [25]. At each step, the planner selects one of the following tools, which correspond to the action space $\mathcal{A}$ defined in Chapter 3:

- `TextRetriever(query)`: dense retrieval over the textual index.
- `TableRetriever(query)`: locates a table through the visual index, then parses it with a table-structure recogniser [22] into structured form, preserving the page coordinates of every cell.
- `FigureRetriever(query)`: visual retrieval over the ColQwen2 page index (of the ColPali family), returning figures together with their patch-similarity scores.

A fourth tool, `RegionZoom`, re-renders a retrieved figure at higher resolution; it is not chosen by the planner but runs automatically when the figure covers less than 30% of its page. Multi-hop navigation (for example, from a textual claim to its supporting table) requires no dedicated tool: it arises from the loop itself, since each retrieved observation informs the query the planner issues next. All tool invocations and their results are recorded in a structured scratchpad, which later serves as the auditable retrieval log referenced in Chapter 1.

4.4 Evidence Sufficiency Gate

After each retrieval round, the Evidence Sufficiency Gate determines whether the evidence accumulated in the scratchpad is sufficient to answer the query. It is implemented as an LLM-as-judge: the prompted model first names the facts in the evidence that answer the question, citing their evidence identifiers, and only then returns its verdict, so that a verdict cannot precede its justification. (A learned gate, a classifier over retrieval-confidence features, was considered and not built: no training labels for sufficiency exist.) If the gate reports insufficient evidence, control returns to the Agentic Planner for an additional retrieval iteration, bounded by a maximum iteration count to prevent unbounded looping. This mechanism directly implements Objective 3 and mitigates hallucination by preventing the system from answering without grounded proof.

4.5 Grounded Generator and Faithfulness Gate

Once the Sufficiency Gate confirms sufficiency, an open-weight vision-language model run locally (Qwen2.5-VL-7B [3], 4-bit) generates an answer strictly conditioned on the retrieved evidence pool; the generator has no other access to the corpus. It writes the answer as a list of claims, each citing its evidence (a quote, a table cell, or a figure observation). The Faithfulness Gate then checks each claim against the evidence it cites, first exactly and then with a support check by the model. Any unsupported claim sends the draft back to the generator for rewriting, again bounded by a maximum iteration count.

4.6 Attribution Layer

Visual attribution prefers exact positions preserved during retrieval and ingestion: the cell coordinates recorded by `TableRetriever`, the bars of vector charts read from the PDF, and the positions of quoted sentences, captions and diagram labels on the page. Only for photographs, which have no printed words, are the patch-level similarity scores of ColQwen2 used to localise the most relevant region. Together these yield bounding-box coordinates over the supporting table cell or figure region. This satisfies Objective 4 and provides the fine-grained visual grounding required for human verifiability (RQ3).

4.7 Mapping to Research Objectives

Table 4.1 summarizes how each architectural component addresses the research objectives introduced in Chapter 1.

Component	**Research Objective**
Ingestion and Indexing Layer	Objective 2
Agentic Planner	Objective 1
Evidence Sufficiency Gate	Objective 3
Grounded Generator and Faithfulness Gate	Objective 3
Attribution Layer	Objective 4

Table 4.1: Mapping of architectural components to research objectives.

Chapter 5

Graph Architecture and Node Specification

This chapter describes the system as a graph: a set of nodes, each with one job, connected by edges that show what flows between them. The goal is that this graph can be built node-for-node in a framework such as LangGraph [16], with nothing left to guess between what the thesis says and what the code does.

5.1 Design Rationale

Two design choices set this system apart from a standard single-pass RAG pipeline.

Retrieval can repeat. A standard pipeline searches once and answers once, so it only works if that one search happens to find everything needed. Here, the planner (N5) can call retrieval tools again and again, and the Sufficiency Gate sends it back until the evidence is actually enough. Multi-hop reasoning is not a prompting trick; it is simply how the loop is built.

Only certain nodes can add evidence. Nodes N6a–N7 are the only ones allowed to bring anything into the evidence buffer (N8), and the generator (N9) can only read from that buffer — it has no other way to see the corpus. This separation is what makes the Faithfulness Gate work: it only has to check claims against one fixed source of truth, instead of guessing whether the generator made something up.

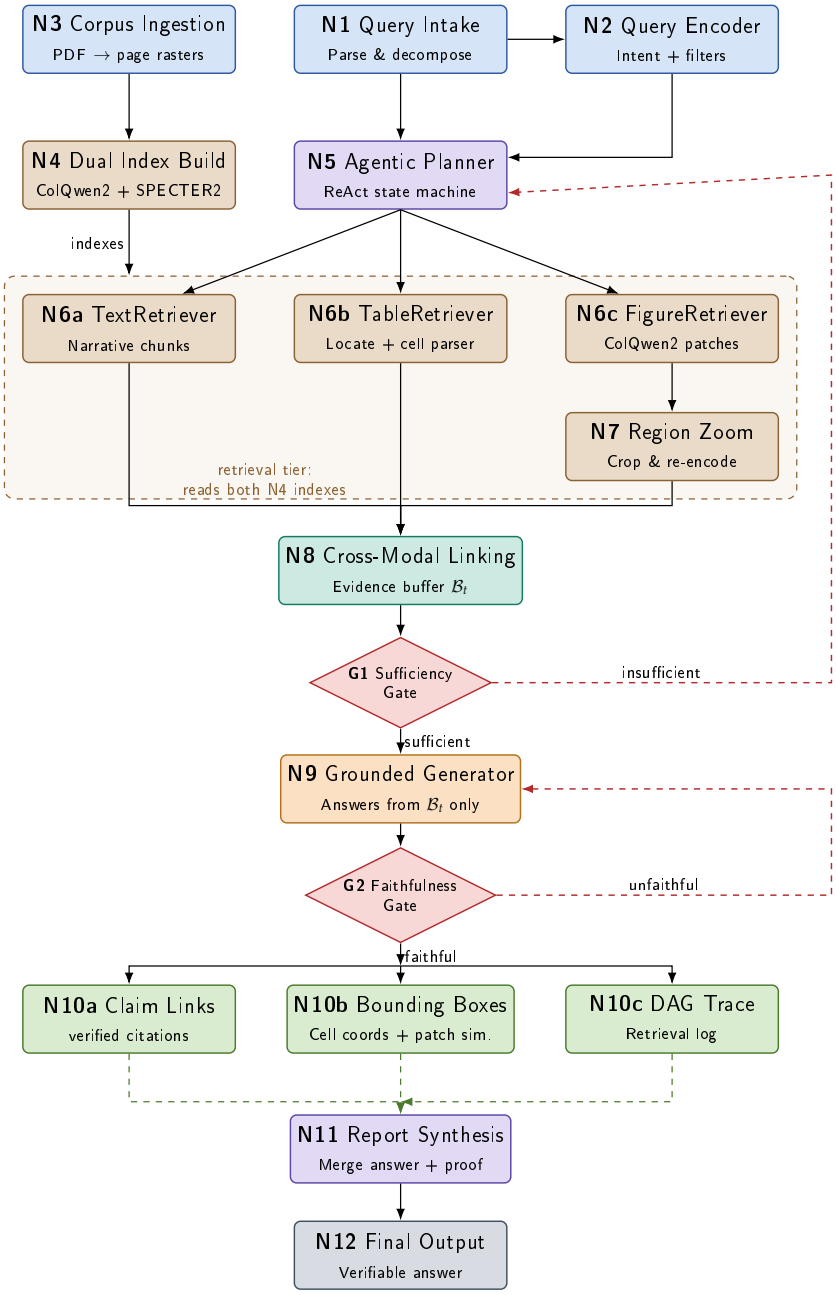


Figure 5.1: Full graph architecture of the proposed Agentic Multimodal RAG system. Blue nodes perform ingestion and query encoding; brown nodes are the retrieval tools that hold exclusive authority to introduce evidence, grouped in the shaded retrieval tier because all of them query the two indexes built by N4; the teal node maintains the evidence buffer that every downstream node reads from; the orange node is the grounded generator; violet nodes are the planner and the synthesiser; green nodes produce the three attribution artefacts. Red diamonds are the two validation gates and red dashed edges are the two correction loops. Green dashed edges carry attribution artefacts forward to synthesis. The planner may traverse the retrieval tier arbitrarily many times before the Sufficiency Gate permits generation.

5.2 Node-by-Node Specification

Here is how one query moves through the graph, start to finish.

> **Running example.** *"Does the reported accuracy gain in Section 4 hold for the low-resource subset, and which table reports it?"*

This question needs two different kinds of evidence: a claim from the text and a number from a table. A single search cannot answer it alone, which is exactly why it is used here.

5.2.1 Ingestion Path (N3–N4)

This part runs once per corpus, offline, before any query arrives.

N3 — Corpus Ingestion. Every PDF page is converted into an image, and its text is separately pulled out and split into chunks. Pages become images rather than parsed layouts because tables are exactly where OCR fails — merged cells and multi-column headers break text extraction. An image keeps the table's structure intact.

N4 — Dual Index Build. Two search indexes are built from the same set of pages: one holds ColQwen2 visual embeddings of each page image (for finding tables and figures), the other holds SPECTER2 text embeddings of each chunk, stored in FAISS [13] (for finding claims). Every retrieval tool (N6a–N6c, N7) reads from these two indexes. Because both indexes point back to the same pages, a match in one can lead directly to the matching spot in the other — which is what makes jumping from a text claim to its supporting table possible.

5.2.2 Query Encoding Path (N1–N2)

N1 — Query Intake. The question is broken into smaller steps before any search happens. For the running example, that is two steps: find the accuracy claim, then find the table that reports the low-resource breakdown. Doing this upfront gives the planner a plan to follow instead of improvising one.

N2 — Query Encoder. The query is embedded, and filters are pulled out of it — here, "Section 4" and "low-resource subset." This has to finish before any retrieval starts, since every search the planner runs afterward depends on these filters.

5.2.3 Planning and Retrieval (N5–N7)

N5 — Agentic Planner. This is the only node that decides what happens next. It keeps track of the plan, what has already been searched, and what evidence has been found, then picks one retrieval tool to run. After seeing the result, it either searches again or moves on to the Sufficiency Gate. It never adds evidence itself — only a retrieval tool can do that.

N6a — TextRetriever. Searches the text index for narrative chunks matching the query. On the running example, this is the first search: it finds the sentence claiming the accuracy gain, along with the page it appears on.

N6b — TableRetriever. Finds a table in two steps: it searches the visual index for pages whose best match is a table, then runs a table-structure recogniser (such as the Table Transformer) on that region to recover rows, columns and the position of every cell. Cell text is read from the PDF at those positions. It keeps track of exactly where each cell sits on the page. This matters because the bounding boxes generated later (N10b) are only possible if this spatial position is kept — a plain-text version of the table would lose it.

N6c — FigureRetriever (ColQwen2). Searches the visual index to find charts, plots, or dense tables without needing OCR. It also keeps the match scores it computed, because those scores are reused later to draw bounding boxes.

N7 — Region Zoom. Re-renders a retrieved figure at higher resolution. This exists because finding the right page and reading a specific data point inside it need different levels of detail — a page-level match tells you which page has the chart, not which point answers the question. It is not chosen by the planner: it runs automatically after N6c whenever the figure covers less than 30% of its page. On the running example the evidence is a table found by N6b, so N7 does not run.

5.2.4 Evidence Buffer and Sufficiency Gate (N8, G1)

N8 — Cross-Modal Linking. Everything retrieved so far is converted into one shared format and added to the evidence buffer, along with where it came from — which page, which tool, which step. Items that share a page, or where one refers to the other (a sentence citing "Table 3" and the table captioned "Table 3"), are linked, so that the generator receives connected evidence rather than a loose

pile. This buffer is the only thing the generator will ever see; nothing after this point looks at the corpus again.

G1 — Sufficiency Gate. Before generation is allowed, a check runs: is there enough evidence here to actually answer the question? On the running example, the first check fails — the text claim is there, but the table that would confirm or deny it for the low-resource subset is not. Control goes back to N5, which fetches the table. Only once both pieces are present does the gate let the process continue.

This check happens *before* generation, not after. A standard pipeline would hand the incomplete evidence to the generator and hope it notices something is missing — which is exactly how confident, wrong answers get produced.

5.2.5 Generation and Faithfulness Gate (N9, G2)

N9 — Grounded Generator. An open-weight vision-language model (Qwen2.5-VL-7B, 4-bit) run locally. It writes the answer as a list of claims, each citing the evidence it rests on — a quote, a table cell, or a figure observation — using only what is in the evidence buffer. It cannot search on its own and cannot fill in numbers from memory — everything it says has to trace back to something already retrieved.

G2 — Faithfulness Gate. Checks every factual claim in the draft answer against the evidence buffer. If the answer states something that is not actually backed by retrieved evidence, the gate fails and N9 is sent back to rewrite that part using only what is really there. Because the buffer is the only source allowed, each claim is checked against one fixed, finite list of retrieved items, rather than against everything the model might believe about the world.

5.2.6 Attribution and Output (N10–N12)

N10a — Claim Links. Links each published claim to the evidence it cites, as verified by G2: the quoted sentence, the table cell, or the figure. This replaces the token attribution by integrated gradients first planned, which cannot be computed through a quantised model served by llama.cpp.

N10b — Bounding Boxes. Draws a box — $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$ — around the exact spot on the page a claim came from. For a table cell, the box is the cell position N6b recorded; for a figure, it is the bar read from the PDF, or the diagram

labels or caption the claim names; for a quote, the sentence found on the page. Only for photographs does it enclose the highest-scoring patches of ColQwen2's similarity map. This only works because N6b kept cell positions and ingestion kept the page layout.

N10c — DAG Trace. Logs what actually happened: which tools ran, in what order, and how many times the Sufficiency Gate sent things back. On the running example, this log shows two search rounds and one rejection — so the process can be checked, not just trusted.

N11 — Report Synthesis. Combines the answer with its three proof artefacts into one readable report.

N12 — Final Output. What the user actually sees: the answer, the highlighted source text, a box on the source page, and the log of how it was found.

5.3 Control Loops

There are exactly two loops in this graph, and each catches a different kind of mistake.

The retrieval loop (not enough evidence). Triggered by the Sufficiency Gate. Sends control back to N5 to search again. This stops the system from confidently answering using incomplete evidence.

The regenerate loop (wrong evidence used). Triggered by the Faithfulness Gate. Sends control back to N9 to rewrite. This stops the system from stating something the evidence does not actually support.

Both loops have a limit on how many times they can repeat. If that limit is reached, the system says the evidence is not enough instead of guessing. For a literature review tool, this is the right trade-off: "I could not confirm this" is useful. A confident wrong answer is not.

5.4 Mapping to Research Objectives

Table 5.1 shows which objective each part of the graph is responsible for.

Nodes	Objective	Contribution
N3, N4, N6c	Objective 2	Vision-document retrieval over rasterised pages
N1, N2, N5, N7	Objective 1	Iterative agentic planning and multi-hop navigation
N8, Sufficiency Gate	Objective 3	Evidence verification prior to generation
N9, Faithfulness Gate	Objective 3	Claim-level grounding of generated output
N10a–N10c, N11	Objective 4	Spatial, textual, and procedural attribution

Table 5.1: Mapping of graph nodes to research objectives.

Chapter 6

Implementation

This chapter describes the system as it was built and run. The graph of Figure 5.1 was implemented node for node in LangGraph [16]; where the implementation departs from the original design, the reason is given.

6.1 Software and Hardware

Table 6.1 lists the components. Everything ran on one desktop PC with a consumer GPU (Intel Arc B580, 12 GB) and 15.6 GB of RAM, fully offline: every model was loaded from a local copy, and no evaluation depended on a paid service.

Component	Choice
Agent framework	LangGraph (Python 3.12), one node per box of Figure 5.1
PDF reading	PyMuPDF [1]: text layer, page images (150 dpi), vector charts
Table structure	Table Transformer [22]
Text index	SPECTER2 [21] (768-d) in FAISS [13], fused with keyword search
Page index	ColQwen2 [10], 128-d vectors per page patch
Language model	Qwen2.5-VL-7B-Instruct [3], 4-bit (Q4_K_M), llama.cpp [12] server
Comparison model	Gemini 3.5 Flash-Lite (free tier), through the same request format

Table 6.1: The software stack.

6.2 Ingestion and Indexing (N3, N4)

Each paper is read once, offline. Its text is split into paragraphs; its tables and figures are located with their captions and bounding boxes; every table is parsed into cells, each with its own page coordinates; and bar charts drawn as vector graphics are read directly from the PDF, so that a bar's value and position are exact rather than estimated from pixels. Two indexes are built: SPECTER2 vectors of the paragraphs in FAISS, and ColQwen2 patch vectors of every page image. Only one large model fits on the 12 GB GPU at a time, so the page index is built before the language model is started, and at question time ColQwen2 runs only from cached question vectors.

6.3 The Language Model in the Loop

The model serves six roles: splitting the question into sub-goals and naming the entities it is about (N1, N2), planning the next tool (N5), judging sufficiency (G1), writing the answer (N9), and checking each claim's support (G2). Every call follows the same pattern:

- The reply is forced into a JSON schema by the server's grammar, so it can always be parsed. Temperature is 0 and the seed is fixed.
- Every reply is cached under a hash of the full request, and the cache is kept under version control. A re-run reproduces every answer exactly, and a power cut never costs more than the question in progress.
- Every reply is capped (2,048 tokens, and length limits inside the schema): the 7B model once repeated itself for 5,800 tokens inside one field.
- If the model cannot be reached or replies without the required fields, the node falls back to a simple rule and the trace records it. Across all test runs, four questions used a fallback.

Two design details mattered more than expected. First, **the Sufficiency Gate writes its facts before its verdict**. llama.cpp fills the fields of a JSON reply in schema order, so a verdict written first was only justified afterwards; with the order reversed, the gate must name the fact that answers the question before declaring sufficiency. Second, **the kind of evidence a sub-goal needs** (table or figure) is not asked of the model but voted by the nine most similar labelled SPIQA questions from papers that are in neither split; on the development questions this chose the reference's kind 17 times out of 20, against 8 for the model.

6.4 Generation and Verification (N9, G2)

The writer receives the evidence buffer — paragraphs, parsed tables as cells, and figure images — and must return a list of claims, each citing its evidence: a verbatim quote, a table cell (row, column, value), or an observation about a figure. The Faithfulness Gate checks every claim in two layers:

1. **Exact check.** A quote must occur in the cited paragraph, and a cell value must equal the parsed cell. Both sides are first normalised for the quirks of PDF text extraction (ligatures, line-end hyphenation, spaced decimals such as "0 . 5", curly quotes); without this, four of the twenty development questions lost a correct claim. A changed word or number still fails.

2. **Support check.** The model judges whether the cited evidence supports the claim as written, which catches a correct quote attached to a wrong conclusion.

A failed claim sends the draft back with the reason, at most twice. Claims still failing are withheld and listed as unconfirmed; if every claim fails, the answer says that nothing can be stated.

6.5 Exact Proof Boxes (N10b)

Every published claim receives a box on the page, chosen in this order: the cited table cell; the bar of a vector chart; the quoted sentence, found on the page (even when split across lines); the caption; the labels inside a diagram that the observation names; and, only for photographs, the box around the highest-scoring patches of ColQwen2's similarity map. For the photograph case, a second pass after the evaluation computed the map for each claim's own text rather than for the question, from cached vectors; 78 claims across the five runs were re-boxed this way, without any change to the answers. Figure 6.1 shows an output.

6.6 Evaluation Runner and Power-Cut Safety

The evaluation ran on a shared PC under daily power cuts, so the runner was built to survive them. Each finished question is appended to a file and forced to disk before the next starts; a restart skips finished questions; a question during which the model became unreachable is discarded and redone, so no stand-in answer enters the results; and every expensive file is written under a temporary

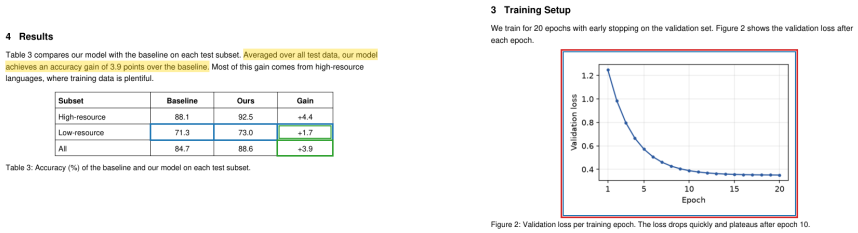


Figure 6.1: A grounded answer: each claim's proof box drawn on the source page.

name and renamed only when complete. One script restarts everything after a power cut: it starts the model server when memory and GPU allow, continues each system where it stopped, and commits the results. On the test split the runner prints no question or answer text and refuses to start if any setting differs from the frozen configuration.

6.7 Design Validation on a Synthetic Paper

Before any real paper was used, the whole graph ran on a small synthetic paper with deterministic stand-ins for every model. This tested the control flow of every path — one round, a zoom, a rejected buffer, a rewritten draft, a draft that is never corrected, an unanswerable question — with exact, repeatable traces (Table 6.2, Figure 6.2). Over 140 automated tests protect this behaviour.

Scenario	R	Z	W	Steps	Node runs	Model calls
Text, 1 round	1	0	0	11	13	6
Figure, zoom	1	1	0	12	14	6
Running example	2	0	0	16	18	8
Running ex., 1 misquote	2	0	1	19	21	10
Running ex., never corrected	2	0	2	23	25	12
Unanswerable	3	0	0	20	20	8
Worst case (3 sub-goals)	3	1	2	29	31	14

Table 6.2: Execution profile on the synthetic paper. R = retrieval rounds, Z = zooms, W = rewrites.



Figure 6.2: Execution trace of the running example on the synthetic paper.

Chapter 7

Results

This chapter reports the evaluation on the 150 test questions, answers each research question in turn, and describes one change that was made during the evaluation. All rates carry 95% bootstrap intervals; differences are paired by question.

7.1 Overview

Figure 7.1 and Table 7.1 summarise the five systems. The full agent answered 21% of the questions correctly, was wrong on 43% and abstained on 35%. Single-pass, which always answers, was correct on 34% and wrong on 66%. The text-only agent was correct on 15%, and the same full graph with Gemini behind it on 30%.

System	Correct	Wrong	Abstained	Prec.	Hit	RF	AES	Search	Calls
Full (agentic, 7B)	21 [15, 28]	43 [35, 51]	35 [28, 43]	33	0.73	0.44	0.44	2.7	11.7
Single-pass	34 [27, 41]	66 [59, 73]	0 [0, 0]	34	0.77	0.41	0.36	3.0	3.0
Text-only agentic	15 [10, 21]	51 [43, 59]	33 [26, 41]	23	0.00	–	0.42	2.8	12.5
Full, strict G1 (ablation)	5 [2, 9]	9 [5, 14]	85 [79, 91]	36	0.73	0.44	0.44	2.7	8.3
Full, Gemini 3.5 Flash-Lite	30 [23, 37]	51 [43, 59]	19 [13, 26]	37	0.65	0.55	0.70	1.9	7.9

Prec. = correct / answered (%). Hit = the reference table or figure was retrieved. RF is undefined for text-only (it retrieves no region). Search = search-tool calls per question; Calls = model calls per question.

Table 7.1: Outcomes on the 150 test questions (%, with 95% bootstrap intervals), precision when answering, retrieval and efficiency. Correct and wrong are judged against SPIQA's gold answer by the model judge (Section 3.7.3); abstained includes answers whose every claim was withheld by G2.

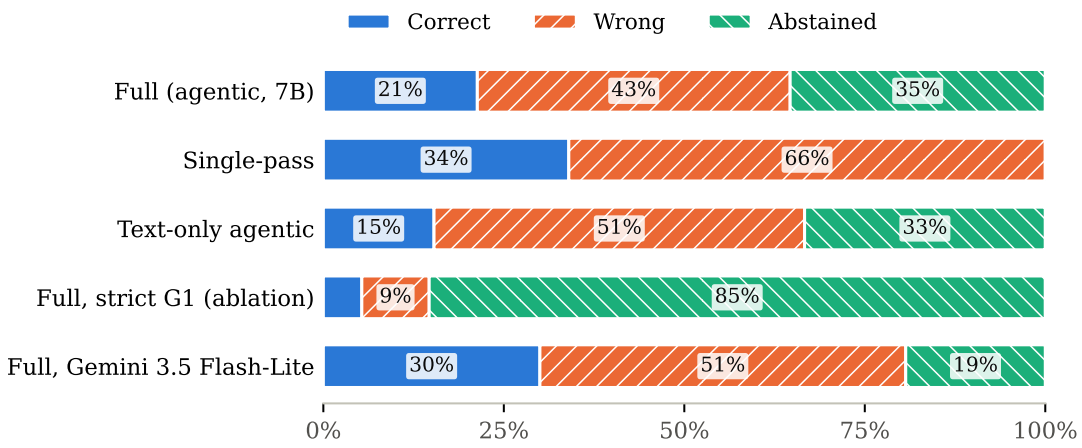


Figure 7.1: Outcomes on the 150 test questions. Correct and wrong as judged against SPIQA's gold answer; abstained includes answers whose every claim was withheld.

The judge. The model judge agreed with the human grader on 22/25 answers (Cohen's kappa [6] 0.72, substantial agreement); the grader marked 5 further answers as undecidable. In all three disagreements the human called the answer correct and the judge wrong; the judge never accepted an answer the human rejected. On the development questions, against earlier hand labels, it agreed on 28 of 34 answers (kappa 0.64), and all six disagreements concerned answers that were only partly right. The judge is therefore reliable and, if anything, strict: the correct rates reported here are conservative for every system, and all wrong rates are high partly because an incomplete answer counts as wrong.

7.2 RQ1: Does the Agentic Loop Improve Accuracy and Reduce Hallucination?

Answer: with the 7B model, the loop and its gates reduce wrong answers substantially, but they do not raise accuracy: they trade correct answers for abstentions, and the abstentions are not selective.

The full agent is wrong 22.7 percentage points less often than single-pass, beyond luck (Table 7.2). It is also correct 12.7 points less often, again beyond luck, because it abstains on 35% of the questions where single-pass answers every one. Its precision when answering is no higher (33% against 34%): the reduction in wrong answers comes from answering less, not from answering better.

Table 7.3 makes this precise. A selective system would hold back the questions that are hard for every system; single-pass's accuracy on those questions

Comparison	Correct	Wrong	Abstained
Full − Single-pass (all 150)	-12.7 [-22, -4]*	-22.7 [-33, -13]*	+35.3 [+28, +43]*
Full − Text-only (all 150)	+6.0 [-2, +14]	-8.0 [-18, +2]	+2.0 [-7, +12]
Full − strict ablation (all 150)	+16.0 [+11, +22]*	+34.0 [+27, +41]*	-50.0 [-58, -42]*
Gemini − Full (7B) (all 150)	+8.7 [+0, +17]	+7.3 [-2, +17]	-16.0 [-25, -7]*
Full − Single-pass (multi-hop 10)	+20.0 [+0, +50]	-40.0 [-70, -10]*	+20.0 [+0, +50]
Full − Text-only (multi-hop 10)	+40.0 [+10, +70]*	+0.0 [-30, +30]	-40.0 [-70, -10]*
Full − strict ablation (multi-hop 10)	+60.0 [+30, +90]*	+20.0 [+0, +50]	-80.0 [-100, -50]*
Gemini − Full (7B) (multi-hop 10)	-20.0 [-60, +20]	+10.0 [-30, +50]	+10.0 [-30, +50]

Table 7.2: Paired differences in percentage points on the same questions, with 95% bootstrap intervals (10,000 resamples). *: the interval excludes 0.

would then be low. For the full agent it is the opposite: single-pass was correct on 40% of the questions the agent held back, and on only 31% of those it answered. With the 7B model, the Sufficiency Gate's doubt does not track how hard a question is. The best-effort flag tells the same story: answers given after the gate stayed unconvinced were correct about as often as those it accepted (24 of 75 against 8 of 22).

System	Held back	Answered
Full (agentic, 7B)	40 (of 53)	31 (of 97)
Full, strict G1 (ablation)	36 (of 128)	23 (of 22)
Full, Gemini 3.5 Flash-Lite	14 (of 29)	39 (of 121)

Table 7.3: Is abstention selective? Single-pass's accuracy (%) on the questions each system held back, and on those it answered. A selective system holds back the questions that are hard for every system.

On the 10 multi-hop questions, where the plan expected the loop to matter most, the picture is more favourable (Table 7.4): the agent was correct on 6 and wrong on 2, against 4 and 6 for single-pass; the difference in wrong answers is beyond luck. With ten questions this is indicative only.

System	Correct	Wrong	Abstained
Full (agentic, 7B)	6	2	2
Single-pass	4	6	0
Text-only agentic	2	2	6
Full, strict G1 (ablation)	0	0	10
Full, Gemini 3.5 Flash-Lite	4	3	3

Table 7.4: Outcomes on the multi-hop subset (10 test questions that ask for a reason or an implication of a result: the result is in a table, the reason in the text). Counts.

The agent is also more economical in the sense of the planned efficiency score: it calls the search tools 2.7 times per question against 3 for single-pass (AES 0.44 against 0.36, beyond luck). It is not cheaper overall: the planner and the gates cost 11.7 model calls per question against 3, and 27 seconds against 11 (medians).

7.3 A Change Made During the Evaluation

The design first frozen for the test run abstained whenever the Sufficiency Gate was not convinced after three rounds. The first test run with it abstained on 110 of the first 142 questions before a power cut stopped it — although the reference table or figure was in the buffer for about two thirds of them: the gate demanded that the evidence state the answer outright, and the 7B model rarely saw it that way.

The plan's rule for a severe problem found during the test run was followed: fix it on the development split only, re-freeze, re-run every system on the test split, and report both runs. The change separates two decisions that the gate had merged. The Sufficiency Gate decides when to *stop searching*; whether a claim may be *published* is the Faithfulness Gate's job. After the last round, if the Sufficiency Gate found facts that answer the question, the agent answers from them as a flagged best effort, and every claim is still checked. On the development questions this moved the full agent from 6 right, 1 partly right, 0 wrong and 13 abstentions to 9, 4, 5 and 2. A second idea, running every tool in the first round as single-pass does, was tried on development and dropped: more evidence produced worse conclusions (7, 5, 6, 2).

No test answer was read while making this change. The strict first design was kept as an ablation and completed on all 150 questions: it abstained on 85%, and of its 22 answers, 8 were correct and 14 wrong.

7.4 RQ2: Can Visual Retrieval Fetch the Right Tables and Figures?

Answer: yes — the reference table or figure reached the evidence buffer for 73% of the questions — but with the 7B model, being able to see tables and figures changed accuracy little, except on multi-hop questions.

Locating the reference works: the agent's buffer held it for 73% of the questions, single-pass's for 77% (single-pass fetches every kind of evidence every time; the agent chooses one per round). On development, ColQwen2 alone ranked the reference's page first for 15 of 20 questions. Of the distinct tables and

figures the agent fetched, 44% were the reference (RF), and 44% of its published claims cite the reference (the automatic stand-in for MAP, Table 7.5).

Against the text-only agent, which reads tables and figures only as extracted text, the full agent was correct 6 points more often and wrong 8 points less often; neither difference is beyond luck (Table 7.2). On the multi-hop subset it was correct 40 points more often, beyond luck. The two systems also abstain about equally (35% and 33%): the text-only agent's abstentions come from the same gates.

7.5 RQ3: Can the Agent Produce Precise Boxes?

Answer: yes, when a claim cites the right table or figure: 71% of such proof boxes land in the hand-drawn answer box, and every table-cell proof of the full agent does (12/12). Per question, however, grounding scores are low, because many questions are abstained and many proofs cite other sources.

System	CGS	CGS ans.	IoU≥0.5 (%)	Point.	Point. ans.	Ref. attr.
Full (agentic, 7B)	0.07	0.12	6	0.16	0.28	0.44
Single-pass	0.15	0.15	6	0.40	0.40	0.42
Text-only agentic	0.00	0.01	0	0.04	0.07	0.00
Full, strict G1 (ablation)	0.03	0.21	2	0.06	0.50	0.61
Full, Gemini 3.5 Flash-Lite	0.05	0.06	2	0.20	0.26	0.55

Table 7.5: Grounding on the 50 questions with hand-drawn answer boxes. CGS: best IoU of a proof box with a hand box (abstention = 0). Pointing: a proof box's centre lies inside a hand box. Ref. attr.: share of published claims citing the reference table or figure (all 150 questions).

Measured by IoU as planned, grounding is poor for every system: the full agent's CGS is 0.07 (Table 7.5). Inspection showed that IoU penalises correct proofs drawn at a different size from the hand box. On one question the agent boxed exactly the two cells the gold answer names, but a parsed cell spans its whole column (43 pt) while the hand box hugs the printed number (23 pt): IoU 0.43 and 0.46, below the usual 0.5 threshold. A correct cell inside a hand box drawn around a whole row scores 0.18. Shrinking every cited cell's box to its printed words did not help (full 0.070 to 0.068): as many questions lost as gained. The pointing game, which asks only whether a proof's centre lies inside a hand box, credits the full agent on 0.28 of its answered questions.

Table 7.6 locates the problem. Of the full agent's 72 proof boxes on the annotated questions, only 31 sit on the reference table or figure; the rest point at paragraphs of text or at other tables. Where a proof does sit on the reference,

System	Proof boxes	On the reference	Inside the answer box	Table cells inside
Full (agentic, 7B)	72	31	22 (71%)	12/12
Single-pass	181	85	53 (62%)	30/44
Full, Gemini 3.5 Flash-Lite	68	28	13 (46%)	6/6

Table 7.6: Where the proof boxes point, on the annotated questions: boxes on the reference table or figure, and how many of those have their centre inside the hand-drawn answer box.

it points at the answer 71% of the time, and its table-cell proofs all do. Single-pass, whose claims no gate checks, puts 30/44 of its cell proofs on the answer. The boxing itself is precise; what limits grounding is upstream — which source a claim cites, and whether the question is answered at all.

7.6 A Stronger Model Behind the Same Graph

With Gemini 3.5 Flash-Lite behind every model call and nothing else changed, the agent was correct on 30%, wrong on 51% and abstained on 19%; its precision when answering was 37%. Its gates behaved differently from the 7B model's: the Sufficiency Gate was convinced on 117 questions instead of 22, after fewer rounds (1.9 against 2.7), and the Faithfulness Gate kept 88% of the drafted claims instead of 50%. Above all, its abstention was selective (Table 7.3): single-pass was correct on only 14% of the questions Gemini held back, against 39% of those it answered. Its retrieval was also more focused (RF 0.55, AES 0.70). The difference in correct answers against the 7B agent, +8.7 points, sits at the edge of the interval ([0, +17]).

Chapter 8

Discussion

8.1 The Model, Not the Architecture, Is the Bottleneck

The central argument of this thesis was that an agent which searches until its evidence suffices, checks every claim, and abstains otherwise, would turn confident errors into stated gaps. With the local 7B model, the first half of this happened and the second did not: the agent was wrong far less often than single-pass, but it held back easy questions about as readily as hard ones, so it also lost correct answers.

Running the identical graph, prompts and settings with a stronger model separates the two explanations. With Gemini, the Sufficiency Gate was convinced about five times as often, the Faithfulness Gate rejected far fewer claims, and — the decisive observation — the questions it held back were the ones single-pass also got wrong. The gates behave as designed when the model behind them can judge sufficiency and read a table or chart reliably. A 4-bit 7B model cannot yet do either well enough: its judgements of sufficiency are not calibrated to difficulty, and half of its drafted claims fail verification. The architecture is therefore not refuted by the 7B results; its benefit is gated by model capacity. This also suggests a practical design for constrained settings: a small local model to read and write, and a stronger model only for the two judgements.

8.2 What the Development Error Analysis Showed

Before the test run, the full agent's answers on the 20 development questions were analysed by hand (Table 8.1). The two most frequent causes were fixed, after which tuning stopped, as planned.

Cause (13 questions not answered correctly, prompt v6)	Count
G2 too strict: a faithful quote rejected for PDF text quirks ("in- ference", "0 . 5%")	4
G1 too lenient: "sufficient" with a fact that does not answer the question	2 (+1)
The writer misread the evidence or answered vaguely	3
The wrong table or figure was chosen	2
A table with a two-level header was read wrongly	1
A computed ratio that cannot be checked against the evidence	1

Table 8.1: Causes of the full agent's errors on the development questions, before the two fixes (normalised quote matching; a Sufficiency Gate that must name the answering fact before its verdict).

The second fix overshot: the stricter gate produced no wrong answer on development but 13 abstentions, and on test it abstained on more than three questions in four. One calibration step was tried and brought wrong answers back without reducing the abstentions, so the stricter gate was kept and the best-effort rule (Section 7.3) was added instead. The lesson generalises: a gate that decides both *when to stop searching* and *whether to answer* is asked to make two different judgements with one threshold.

8.3 Measuring Grounding

IoU against a hand-drawn box is the natural measure of box precision, and it was fixed before the evaluation. It turned out to measure the agreement of two drawing conventions as much as the precision of a proof: a parsed table cell spans its column, a human box hugs the number, and a human asked to box "the answer" to a comparison may box a whole row. The pointing game is insensitive to these conventions and was therefore added; it is reported beside IoU, not instead of it, and it was added after the IoU results were seen. A cleaner protocol would fix the unit of annotation in advance — for tables, whole cells — so that both sides draw at the same granularity.

8.4 Practical Constraints

The system was built and evaluated under three constraints that shaped it. One consumer GPU with 12 GB of memory holds one large model at a time, so the page index was built before the language model started and ColQwen2 ran at question time only from cached vectors. Daily power cuts made every long job resumable and every file write atomic, and the model-reply cache, kept under version control, made a power cut cost at most one question. The PC was shared with other experiments, so every model load first checked free memory and GPU memory, after an early crash of another job when memory ran out. The stronger-model comparison used a free tier and took three sittings because of rate limits and busy servers; the last 25 questions used a second free-tier key, with the model, prompts and settings unchanged.

8.5 Threats to Validity

- **The judge is a model**, and the same model as the evaluated local systems. Its agreement with a human grader was substantial (kappa 0.72 on 25 answers), and it erred towards strictness, but 30 graded answers is a small check.
- **One annotator**, who is a co-author and knew the project's aims, produced the boxes and the grades; the first quality-control round was redrawn by the first author. The 50 annotated questions bound the precision of every grounding number.
- **Post-hoc decisions.** The best-effort rule was introduced after a first test run, on development data only, and all systems were then re-run; the pointing game was added after the IoU results. Both are disclosed, and the original measures are reported.
- **Scope.** 150 questions from 31 computer-science papers; retrieval within a single known paper rather than across a corpus; a 4-bit model; one run per system at temperature 0. The multi-hop subset has ten questions.
- **MAP** was measured by an automatic stand-in that undercounts correct citations of supporting text.
- **Verifiability by humans** (RQ3's second half) was argued from the precision of the proofs, not measured with a user study.

Chapter 9

Conclusion

This thesis built and evaluated an agentic multimodal retrieval-augmented generation system for questions about scientific papers. A planner fetches text, tables and figures over several rounds; a Sufficiency Gate decides when the evidence suffices; a Faithfulness Gate verifies every claim against its cited evidence; and every published claim receives a proof box on the page and a place in an execution trace. The system was evaluated on 150 SPIQA questions against a single-pass and a text-only baseline, with a local 4-bit 7B model and, behind the identical graph, a stronger hosted model.

RQ1. With the 7B model, the agentic loop with its gates was wrong on 43% of the questions against 66% for single-pass, a reduction beyond luck; but it was also correct less often (21% against 34%), because its abstentions did not single out the hard questions. On the small multi-hop subset it was both less often wrong and more often right. With a stronger model behind the same graph, abstention became selective.

RQ2. Visual retrieval located the reference table or figure for 73% of the questions. Seeing tables and figures rather than their extracted text changed overall accuracy little with the 7B model, but helped clearly on multi-hop questions.

RQ3. When a claim cited the right table or figure, its proof box pointed at the answer 71% of the time, and every checked table-cell proof did. Grounding per question was limited by abstentions and by claims citing other sources, and IoU

proved a poor yardstick when proof and annotation are drawn at different granularity.

9.1 Contributions

- An agentic multimodal RAG design with two gates and verified claim links, implemented end to end and run on consumer hardware with an open model.
- Exact proof boxes — table cells, chart bars read from the PDF, quoted sentences, captions and diagram labels — with the retriever's similarity map kept only for photographs.
- A three-way evaluation (correct, wrong, abstained) with a checked model judge, a selectivity analysis of abstention, and a grounding evaluation on hand-annotated answer regions.
- Evidence that the benefit of the gates depends on the capacity of the model behind them, obtained by holding the architecture fixed and changing the model.

9.2 Future Work

The results point to three directions. First, a hybrid in which the small local model reads and writes while a stronger model makes the two judgements; it should be tested on fresh questions from the unused SPIQA papers. Second, steering claims to cite the table or figure that holds the answer rather than surrounding text, which would raise grounding directly. Third, a user study measuring whether proof boxes and traces actually help researchers verify answers, and an extension beyond computer-science papers.

Bibliography

[1] Artifex Software. PyMuPDF. https://github.com/pymupdf/PyMuPDF, 2024.

[2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.

[3] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[4] Iz Beltagy, Kyle Lo, and Arman Cohan. SciBERT: A pretrained language model for scientific text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[5] Jaemin Cho, Debanjan Mahata, Ozan Irsoy, Yujie He, and Mohit Bansal. M3DocRAG: Multi-modal retrieval is what you need for multi-page multi-document understanding. *arXiv preprint arXiv:2411.04952*, 2024.

[6] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[7] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

[8] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[9] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAs: Automated evaluation of retrieval augmented generation. In *Proceed-*

ings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, pages 150–158, 2024.

[10] Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. ColPali: Efficient document retrieval with vision language models. In *International Conference on Learning Representations (ICLR)*, 2025.

[11] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6465–6488, 2023.

[12] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp, 2023.

[13] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.

[14] Amita Kamath, Robin Jia, and Percy Liang. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 5684–5696, 2020.

[15] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020.

[16] LangChain. LangGraph. https://github.com/langchain-ai/langgraph, 2024.

[17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[18] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2263–2279, 2022.

[19] Minesh Mathew, Dimosthenis Karatzas, and C. V. Jawahar. DocVQA: A dataset for VQA on document images. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 2200–2209, 2021.

[20] Shraman Pramanick, Rama Chellappa, and Subhashini Venugopalan. SPIQA: A dataset for multimodal question answering on scientific papers. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[21] Amanpreet Singh, Mike D'Arcy, Arman Cohan, Doug Downey, and Sergey Feldman. SciRepEval: A multi-format benchmark for scientific document representations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[22] Brandon Smock, Rohith Pesala, and Robin Abraham. PubTables-1M: Towards comprehensive table extraction from unstructured documents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[23] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

[24] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024.

[25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[26] Shi Yu, Chaoyue Tang, Bokai Xu, Junbo Cui, Junhao Ran, Yukun Yan, Zhenghao Liu, Shuo Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. VisRAG: Vision-based retrieval-augmented generation on multi-modality documents. *arXiv preprint arXiv:2410.10594*, 2024.

[27] Jianming Zhang, Sarah Adel Bargal, Zhe Lin, Jonathan Brandt, Xiaohui Shen, and Stan Sclaroff. Top-down neural attention by excitation backprop. *International Journal of Computer Vision*, 126(10):1084–1102, 2018.

[28] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with

MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.